SAKARYA
ÜNİVERSİTESİ

2025-2026 Spring

Operating Systems Course Project

PROJECT TEAM

Rana İrem Özen	-	B241202002
Elif Gül Arslan	-	B231202061
Ayşenur Yılmaz	-	B231202019
Vildan Karaca	-	B231202027

Instructor: Prof.Dr. ÜNAL ÇAVUŞOĞLU

Submission Date: 15.05.2026

a. Introduction

This report presents the design, implementation, and performance evaluation of a multithreaded Producer-Consumer simulation. The primary objective of this project is to build a concurrent system where multiple producer and consumer threads interact with shared bounded buffers safely. To prevent race conditions and ensure data integrity, robust synchronization mechanisms, specifically mutex locks and semaphores, were implemented. Additionally, a dedicated monitor thread was developed to detect and report system deadlocks in real-time. Overall, this project demonstrates the practical application of core operating systems concepts, including thread lifecycle management, inter-process synchronization, and deadlock handling.

b. Design Approach

Our design follows a modular and dynamic architecture. Instead of hardcoding the variables, the system initializes by parsing a configuration file (config.txt). This parser dynamically determines the sizes of the shared buffers, the number of producers and consumers, their routing rules, and their execution intervals.

- **Thread Management:** We utilized standard POSIX threads (pthreads) to create independent execution flows for each producer and consumer defined in the configuration.

- **Shared Resources:** The bounded buffers act as shared memory spaces where producers insert items and consumers extract them.

- **Monitoring System:** To ensure system reliability, a background monitor thread runs concurrently. This thread periodically checks the state of all active threads to identify any starvation or deadlock situations by tracking their waiting times.

- **Memory Safety:** The parser features implicit memory handling. If a specific buffer is not required for a given experiment, the system automatically allocates a safe, size-1 dummy buffer in the background to prevent memory violations (Segmentation Faults) without requiring manual configuration.

c. Synchronization Design

In this project, multiple producer and consumer threads operate concurrently on shared memory spaces (Buffer A and Buffer B). To prevent race conditions and ensure thread-safe operations, we implemented a robust synchronization mechanism using a combination of **Mutex Locks** and **Semaphores**.

1. Mutual Exclusion (Mutex)

When a producer wants to add an item to a buffer, or a consumer wants to remove one, they must access shared variables (like the buffer index). If two threads modify these variables simultaneously, data corruption occurs. To prevent this, we used `pthread_mutex_t`.

Before any thread accesses the critical section (the buffer), it must acquire the lock. Once the operation is complete, the lock is released for other threads.

```
pthread_mutex_lock(&buffer_mutex);
// Critical Section: Add or remove item from the buffer
pthread_mutex_unlock(&buffer_mutex);
```

2. State Synchronization (Semaphores)

While mutexes protect the data, we also needed to control the flow of execution based on the buffer's state (empty or full). For this, we utilized two POSIX semaphores for each buffer: `empty_count` and `full_count`.

- **Producer Logic:** A producer must wait (`sem_wait`) if the buffer is completely full (`empty_count == 0`). After successfully adding an item, it signals (`sem_post`) the `full_count` semaphore to notify waiting consumers.
- **Consumer Logic:** Conversely, a consumer must wait if the buffer is entirely empty (`full_count == 0`). After consuming an item, it signals the `empty_count` semaphore to notify waiting producers.

```
// Producer side
sem_wait(&empty_count);  // Wait if buffer is full
pthread_mutex_lock(&mutex); // Lock the buffer
```

```
// Insert Item
pthread_mutex_unlock(&mutex); // Unlock the buffer
sem_post(&full_count);   // Signal that buffer has
a new item
```

This dual-layered approach (Mutex + Semaphores) guarantees that the system is fully synchronized, thread-safe, and capable of handling high loads without data loss.

d. Deadlock Detection Method

In a complex multithreaded environment, especially under high load or bottleneck scenarios, threads can potentially enter a state of deadlock or severe starvation, waiting indefinitely for a resource that will never become available. To proactively identify such critical failures, we implemented a dedicated **Monitor Thread**.

The deadlock detection mechanism operates based on waiting times:

- **Timestamp Tracking:** Whenever a producer or consumer thread attempts to acquire a lock (via sem_wait or pthread_mutex_lock) and is forced to wait, it records the exact timestamp of when its waiting period began.
- **Periodic Scanning:** The Monitor Thread runs continuously in the background, waking up at regular intervals (e.g., every 2 seconds) to scan the status of all active threads.
- **Threshold Evaluation:** During the scan, the monitor calculates the total elapsed waiting time for each blocked thread. If a thread has been in a waiting state for a duration exceeding a predefined safety threshold (e.g., 5 seconds), the monitor officially flags the situation as a **Deadlock**.
- **Reporting:** Upon detecting a deadlock, the monitor immediately prints a warning to the console, indicating which specific thread is stuck and for how long, allowing for accurate performance evaluation and debugging.

This time-based detection approach ensures that the system does not hang infinitely and provides real-time feedback on the health of the synchronization logic.

e. Experiments

In order to comprehensively evaluate the reliability of our synchronization logic and the accuracy of the monitor thread, we designed various test scenarios. By modifying the config.txt file, we tested the system under different workloads and stress conditions. To ensure consistency and allow for fair comparison across all metrics, each experiment was configured to run for exactly 60 seconds (t:60). The following experimental setups were established:

➢ Experiment 1: Low Load (Ideal Condition)

The goal of this experiment is to observe the system's baseline behavior without any resource stress.

- **Buffer Sizes:** Buffer A = 6
- **Thread Counts:** 2 Producers, 2 Consumers
- **Execution Times:** 10ms for both producers and consumers (Balanced intervals).
- **Expected Outcome:** Smooth data flow, empty/full counts remaining balanced, and no deadlocks.

➢ Experiment 2: High Load (Stress Test)

The goal of this experiment is to observe the system's behavior and stability when subjected to a massive number of concurrent threads.

- **Buffer Sizes:** Buffer A = 20
- **Thread Counts:** 10 Producers, 10 Consumers
- **Execution Times:** Balanced but intensive (10ms interval for producers, 10ms duration for consumers).
- **Expected Outcome:** High CPU utilization and frequent context switching. However, due to the large buffer size, the system should handle the heavy traffic without crashing or losing data.

➢ Experiment 3: Deadlock (Artificial Induction)

The primary objective of this setup is to intentionally force the system into a deadlock state to verify the accuracy of the Monitor Thread.

- **Buffer Sizes:** Buffer A = 5, Buffer B = 5
- **Thread Counts:** 4 Producers, 5 Consumers

- **Execution Times:** 10ms for both producers and consumers.
- **Expected Outcome:** By designing the thread structure to artificially induce a circular wait, the system will freeze. The Monitor Thread is expected to successfully detect this and print a deadlock warning.

➢ Experiment 4: Bottleneck (Producer Slowdown)

This scenario is designed to test how the system handles a situation where data consumption is much faster than data generation.

- **Buffer Sizes:** Buffer A = 5
- **Thread Counts:** 4 Producers, 10 Consumers
- **Execution Times:** Producers are intentionally slowed down (50ms interval) compared to consumers (10ms duration).
- Buffer A will frequently remain empty. The numerous consumer threads will continuously block (sem_wait), creating a severe bottleneck, though not necessarily a complete deadlock.

➢ Experiment 5: Circular Dependency (Complex Routing)

The goal of this final experiment is to observe system stability under complex routing rules, circular dependencies, and extreme speed variations.

- **Buffer Sizes:** Buffer A = 5, Buffer B = 5
- **Thread Counts:** 2 Producers, 10 Consumers
- **Execution Times:** Producers are extremely slow (100ms) while consumers are fast (10ms).
- **Routing Logic:** All producers feed Buffer A. 2 consumers move data from A to B. 6 consumers move data from B to A. 2 consumers consume from A only.
- **Expected Outcome:** The complex cross-routing combined with the extreme speed difference will create heavy starvation. It is highly likely to trigger the deadlock detection mechanism due to the circular dependency between Buffer A and Buffer B.

f. Performance Evaluation

This section presents the empirical results obtained from executing the five experimental scenarios. The metrics collected evaluate the efficiency of our synchronization mechanisms and the accuracy of the deadlock detection monitor.

➢ Experiment 1: Low Load Results

```
============ PERFORMANCE METRICS ============
Total Deadlocks Detected: 0
-----------------------------------------------
THREAD ID   | THROUGHPUT      | BLOCKING TIME (ms)   | AVG WAIT (total wait/item) (ms)
-----------------------------------------------
P1          | 5878            | 53.38                | 0.01
P2          | 5879            | 60.91                | 0.01
C1          | 5878            | 149.43               | 0.03
C2          | 5878            | 134.84               | 0.02
===============================================
```

Throughput: Approximately 11,750 items produced and 11,750 items consumed in total.
Max Wait Times: Producer max blocking = 60.91 ms, Consumer max blocking = 149.43 ms.
Deadlocks Detected: 0
Analysis: Under ideal conditions, the system demonstrated stable performance. The mutex and semaphore implementations successfully prevented race conditions without causing any noticeable delays. The buffer remained balanced.

➢ Experiment 2: High Load Results

```
============ PERFORMANCE METRICS ============
Total Deadlocks Detected: 0
-----------------------------------------------
THREAD ID   | THROUGHPUT      | BLOCKING TIME (ms)   | AVG WAIT (total wait/item) (ms)
-----------------------------------------------
P1          | 5905            | 92.08                | 0.02
P2          | 5904            | 87.24                | 0.01
P3          | 5904            | 87.68                | 0.01
P4          | 5899            | 67.72                | 0.01
P5          | 5905            | 97.85                | 0.02
P6          | 5902            | 55.28                | 0.01
P7          | 5905            | 100.10               | 0.02
P8          | 5904            | 84.52                | 0.01
P9          | 5904            | 83.54                | 0.01
P10         | 5903            | 95.57                | 0.02
C1          | 5900            | 116.49               | 0.02
C2          | 5903            | 146.28               | 0.02
C3          | 5904            | 132.70               | 0.02
C4          | 5903            | 132.31               | 0.02
C5          | 5904            | 141.05               | 0.02
C6          | 5904            | 135.18               | 0.02
C7          | 5904            | 129.74               | 0.02
C8          | 5902            | 136.73               | 0.02
C9          | 5903            | 136.33               | 0.02
C10         | 5901            | 127.19               | 0.02
```

Throughput: Approximately 59,000 items produced and 59,000 items consumed in total.
Max Wait Times: Producer max blocking = 100.10 ms, Consumer max blocking = 146.28 ms.
Deadlocks Detected: 0
Analysis: Despite the heavy influx of 20 concurrent threads, the system maintained complete thread safety. Context switching was much more frequent, leading to slightly higher total blocking times compared to the low load scenario. However, the generous buffer size (A=20) absorbed the heavy traffic perfectly, preventing the system from freezing or losing data. This proves the scalability of our synchronization design.

➢ Experiment 3: Deadlock Results

```
============ PERFORMANCE METRICS ============
Total Deadlocks Detected: 243
-----------------------------------------------
THREAD ID   | THROUGHPUT      | BLOCKING TIME (ms)   | AVG WAIT (total wait/item) (ms)
-----------------------------------------------
P1          | 3               | 0.18                 | 0.06
P2          | 4               | 1.70                 | 0.42
P3          | 4               | 0.01                 | 0.00
P4          | 4               | 1.58                 | 0.40
C1          | 4               | 11.69                | 2.92
C2          | 3               | 12.24                | 4.08
C3          | 3               | 11.91                | 3.97
C4          | 3               | 11.97                | 3.99
C5          | 2               | 22.20                | 11.10
===============================================
```

Throughput: System effectively halted (Average of 3-4 items per thread).
Max Wait Times: Threads exceeded the 5000ms threshold repeatedly.
Deadlocks Detected: 243
Analysis: By implementing a strict circular wait dependency between Buffer A and Buffer B with minimal buffer sizes, a complete resource deadlock was successfully induced. The Monitor Thread performed flawlessly, detecting that threads were unable to proceed for over 5 seconds. The high number of "Deadlocks Detected" (243) reflects cumulative reporting: with scans every 2 seconds over 60 seconds (~30 scans), multiple blocked threads were flagged per scan. This validates the reliability of our deadlock detection and monitoring module.

➢ Experiment 4: Bottleneck Results

Throughput: ~4,772 items produced and ~4,772 items consumed in total.
Max Wait Times: Producers were rarely blocked (max ~24.52 ms), whereas consumers suffered massive blocking times (up to ~55,132.99 ms).
Deadlocks Detected: 0

Analysis: This scenario successfully demonstrated a severe resource bottleneck. Because the production interval (50ms) was significantly slower than the consumption interval (10ms) across a large pool of consumers, the consumer threads spent almost their entire execution time (over 55 seconds of the 60-second runtime) in a blocked state (sem_wait), desperately waiting for Buffer A to be replenished. This "Consumer Starvation" confirms that the system's overall throughput is bottlenecked by the slowest component in the pipeline.

```
============ PERFORMANCE METRICS ============
Total Deadlocks Detected: 0
-----------------------------------------------
THREAD ID   | THROUGHPUT      | BLOCKING TIME (ms)   | AVG WAIT (total wait/item) (ms)
-----------------------------------------------
P1          | 1193            | 16.72                | 0.01
P2          | 1193            | 24.52                | 0.02
P3          | 1193            | 23.35                | 0.02
P4          | 1193            | 20.85                | 0.02
C1          | 477             | 55027.07             | 115.36
C2          | 478             | 55016.78             | 115.10
C3          | 477             | 55077.56             | 115.47
C4          | 478             | 55114.37             | 115.30
C5          | 477             | 55077.64             | 115.47
C6          | 477             | 55076.63             | 115.46
C7          | 477             | 55075.61             | 115.46
C8          | 477             | 55126.59             | 115.57
C9          | 478             | 55117.80             | 115.31
C10         | 476             | 55132.99             | 115.83
===============================================
```

➢ Experiment 5: Circular Dependency Results

Throughput: Initial producers generated ~598 items. Consumers experienced heavy variations (between ~199 to ~598 items).

Max Wait Times: Massive starvation observed, with most consumers (C1 through C10) waiting over 53,000 ms, peaking at ~57,933 ms (C3).

Deadlocks Detected: 0

Analysis: Although the complex thread routing introduced a theoretical circular dependency (some consumers routing A to B, while others routed B to A), an actual system deadlock did not occur. This is because the primary producers (P1 and P2) were significantly slower (100ms) than the consumers (10ms). As a result, the buffers never reached maximum capacity to trigger the necessary "circular wait" condition for a deadlock. Instead, the system suffered from extreme **Consumer Starvation**. The fast consumers instantly depleted the buffers and spent almost the entire execution time (up to 57 seconds) blocked, waiting for new items to be produced rather than deadlocking. This revealed that circular dependency alone is insufficient for deadlock — buffers must also reach full capacity, a key distinction between starvation and deadlock.

```
============= PERFORMANCE METRICS =============
Total Deadlocks Detected: 0
-----------------------------------------------
THREAD ID   | THROUGHPUT      | BLOCKING TIME (ms)   | AVG WAIT (total wait/item) (ms)
-----------------------------------------------
P1          | 598             | 3.49                 | 0.01
P2          | 598             | 8.12                 | 0.01
C1          | 598             | 53773.64             | 89.92
C2          | 598             | 53786.40             | 89.94
C3          | 200             | 57933.07             | 289.67
C4          | 200             | 57929.22             | 289.65
C5          | 199             | 57736.49             | 290.13
C6          | 199             | 57743.36             | 290.17
C7          | 199             | 57837.91             | 290.64
C8          | 199             | 57845.92             | 290.68
C9          | 598             | 53916.18             | 90.16
C10         | 598             | 53916.70             | 90.16
===============================================
```

g. Conclusion

In conclusion, this project successfully implemented a multi-threaded synchronization system using POSIX threads, mutexes, and semaphores in a Linux environment. The dynamic parsing of the config.txt file allowed for extensive testing across various workloads. Through the five designed experiments, we effectively demonstrated key Operating Systems concepts, including ideal low-load conditions, heavy context switching under high load, severe performance bottlenecks (starvation), and actual resource deadlocks. Furthermore, the implementation of an independent Monitor Thread proved highly effective in accurately detecting system freezes without falsely flagging slow operations. Ultimately, this project highlighted the critical balance required in thread synchronization to maintain both thread safety and system efficiency.